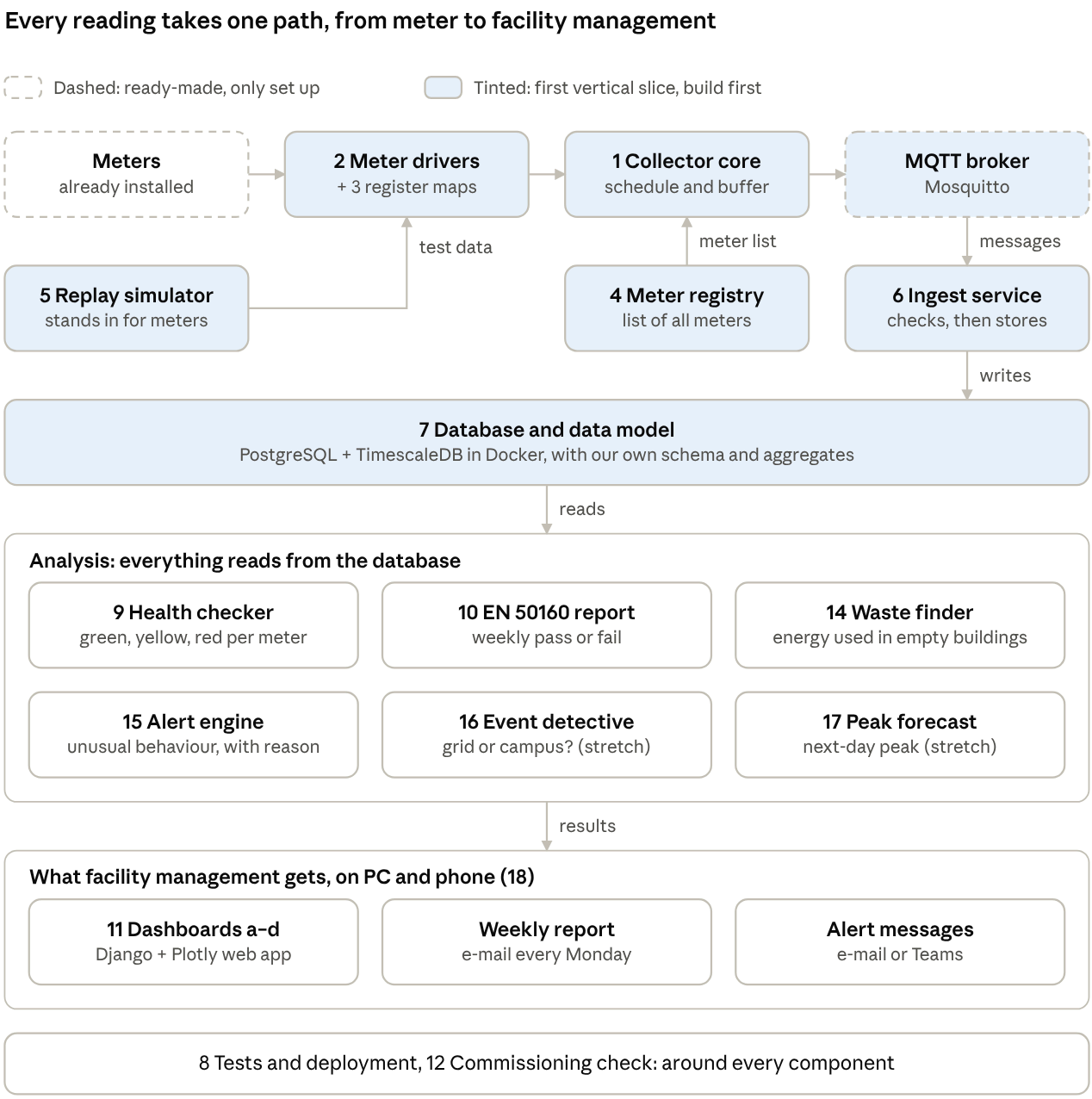
GM Campus Power Monitoring: Components Explained

Oct 28, 2026 · Gaurav | Krishnapriya | Hanna

How the system fits together

Every reading takes one path: a driver reads the meter at the edge, the collector publishes it over MQTT, the ingest service stores it, and all analysis, reports, alerts and dashboards read from that one database.



system architecture · 17 components, numbered as below

The tinted boxes form the first vertical slice: build them first, then add the analysis boxes one at a time.

Each section below follows the same pattern: why the component is needed, how it works, a worked example, what you learn, and a "done when" test so the team knows when to tick it off. Addresses, IDs, building names and numbers in the examples are invented for illustration; real values come from the meter manual and your own data.

Design goal: adding a meter of a supported type needs no code change anywhere; adding a new protocol needs only one new driver.

Second design goal: every screen facility management uses also works on a phone (component 18).

#	Component	What it means	Priority	Status
1	Collector core	Reads each meter on a fixed schedule, sends the values to MQTT, buffers them if the network drops	Must	Not started
2	Meter drivers	One small module per protocol that talks to the meter and returns values with standard names	Must	Not started
3	Register map files	One file per meter model: which register holds which value, its type, unit and scaling	Must	Not started
4	Meter registry	The one list of all meters: where they are, how to reach them, which meter feeds which	Must	Not started
5	Replay simulator	Plays recorded data back as if live, with optional faults, so you can test without the real meter	Must	Not started
6	Ingest service	Takes messages from MQTT, checks them, and writes the good ones to the database	Must	Not started
7	Data model	How data is stored: tables, UTC timestamps, how long raw data is kept, 1/10/15-min averages	Must	Not started

#	Component	What it means	Priority	Status
8	Tests and deployment	Automatic tests on every change; one command starts the whole system	Must	Not started
9	Data health checker	Flags missing, frozen or impossible data: green, yellow or red per meter	Must	Not started
10	EN 50160 evaluator and weekly report	Checks last week's voltage quality against EN 50160 and e-mails a pass/fail report	Must	Not started
11	Dashboards a–d	Four screens: campus overview, one meter in detail, meters side by side, periods side by side	Must (a, b), Should (c, d)	Not started
12	Commissioning check and guide	Checks a new meter's readings before they are trusted, plus a step-by-step guide for adding meters	Should	Not started
13	User management	Personal accounts with Admin, Operator or Viewer rights	Should	Not started
14	Waste finder	Finds energy used when buildings are empty and estimates what could be saved	Should	Not started
15	Alert engine	Warns when a meter behaves unlike its usual pattern, and says why	Should (rules), Stretch (ML)	Not started
16	Event detective	Tells whether a voltage dip came from the public grid or from inside the campus	Stretch	Not started

#	Component	What it means	Priority	Status
17	Peak forecast	Predicts tomorrow's load and warns before a new yearly power peak	Stretch	Not started
18	Mobile access	The web app installs as a PWA on phone and PC: own icon, full-screen, push alerts; overview, alerts, reports and the commissioning check	Should (design for it from the start)	Not started
19	Data export and API	Lets people download chosen measurements as CSV, or fetch them read-only from a web address, without touching the database	Should	Not started

Not developed by the team, only set up: the MQTT broker (Mosquitto), the PostgreSQL database (running in Docker) and standard Python libraries.

The tools are already fixed in the team's Development Setup Guide: Python 3.12, a Django web app with Plotly charts instead of Grafana, APScheduler for timed jobs, and pymodbus, asyncua and paho-mqtt for meters and messaging. Requirements and the session roadmap are in Vision, Ideas & Blueprint; phone access and login are in Phone Access and Login.

1. Collector core

The collector core is the always-running program at the edge that decides when each meter is read and turns every reading into a standard message on MQTT.

Why we need it. Meters only answer when asked, so something has to ask them on a fixed schedule, all day, every day. Keeping this logic apart from protocol details means it is written once and reused for every meter.

How it works.

1. Load the meter registry (component 4) at start-up.

2. For each meter, call its driver's `read()` on the meter's schedule, for example every 1 s for voltage, current and power.
3. Attach the meter ID and a UTC timestamp taken at the moment of reading.
4. Publish the message to MQTT.
5. If a meter does not answer, log it, retry with growing waits (1 s, 2 s, 4 s, up to 60 s), and keep reading all other meters.
6. If the broker is unreachable, write messages to a local buffer file (for example SQLite) and send them in order once the connection is back.

Example message.

```
{
  "meter_id": "gm-main-01",
  "ts": "2026-11-04T09:15:00.000Z",
  "values": {
    "voltage_l1_v": 231.2,
    "current_l1_a": 42.7,
    "active_power_l1_kw": 9.38,
    "thd_voltage_l1_pct": 2.1
  }
}
```

Example scenario. The campus network drops from 09:00 to 09:20. At one reading per second, the collector buffers 1,200 messages locally and sends them after reconnecting. The database shows no gap.

Pitfall: timing drift. Writing `read(); sleep(1)` drifts, because the read itself takes time. If it takes 0.2 s, you get one reading every 1.2 s. Schedule against the clock instead: next reading at 09:15:01.000, then 09:15:02.000, and so on.

What you learn. Long-running programs, scheduling, logging and failure handling.

Done when. It runs 48 hours unattended against the replay simulator and survives a broker restart without losing a message.

2. Meter drivers

A driver is a small module that talks one protocol and returns values with standard names and units, so nothing else in the system needs to know which meter type it is.

Why we need it. Meters speak different protocols. Wrapping each one in a driver with the same interface is what makes "new protocol = one new driver" true.

The interface. Every driver has the same three methods:

```
class MeterDriver:
    def connect(self): ...
    def read(self) -> dict: ...   # {"voltage_l1_v": 231.2, ...}
    def close(self): ...
```

The first driver is `modbus_tcp`. Later ones could be `modbus_rtu` (RS-485 bus), `http_json` (meters with a web API) or `opc_ua`.

The team's package list already includes pymodbus for Modbus and asyncua for OPC UA, because the campus meters may speak either.

Modbus in five points.

- The collector is the client; the meter is the server, reached by IP address, port 502 and a unit ID.
- Data sits in numbered 16-bit registers. Holding registers are read with function code 03, input registers with 04; the manual says which your meter uses.
- One register holds 16 bits, so a 32-bit float (most measured values) spans two neighbouring registers.
- Read neighbouring values in one request (a block), not one request per value. It is faster and loads the meter less.
- Byte order and word order differ between manufacturers. The manual states it; a test confirms it.

Example: decoding a voltage. The meter returns two registers, `0x4367` and `0x3333`. Joined in big-endian order they form the float 231.2 V:

```
import struct

regs = [0x4367, 0x3333]
raw = struct.pack(">HH", *regs)          # bytes 43 67 33 33
voltage = struct.unpack(">f", raw)[0]    # 231.2 after rounding
```

Pitfall: wrong word order. Joined the other way round (`0x3333`, `0x4367`), the same registers decode to about 0.00000004. A value that tiny, or absurdly large, almost always means the word order is wrong.

What you learn. Industrial protocols, binary data, and designing against an interface.

Done when. The Modbus driver returns every value your meter offers under standard names, and unit tests decode known register samples correctly.

3. Register map files

A register map file describes one meter model: which register holds which value, in which data type, unit and scaling.

Why we need it. Register layouts change between models and even firmware versions. Keeping them in data files instead of code means a new model with a known protocol needs a new file, not new code.

What each entry holds. Standard value name, register address, data type (float32, int16, uint32, int64), unit, and an optional scale factor.

Example (addresses invented; take the real ones from your meter's manual):

```
model: Example Meter X
protocol: modbus_tcp
function_code: 3
word_order: big
blocks:
  - start: 0
    count: 40
  - start: 256
    count: 2
values:
  - name: voltage_l1_v
    address: 0
    type: float32
    unit: V
  - name: current_l1_a
    address: 12
    type: float32
    unit: A
  - name: active_energy_import_kwh
    address: 256
    type: uint32
    scale: 0.01
    unit: kWh
```

In this example, a raw energy count of 1234567 means 12,345.67 kWh.

Naming convention. Use quantity_phase_unit in lowercase, for example `voltage_l1_v`, `current_n_a`, `thd_voltage_l2_pct`, `flicker_plt_l3`, `frequency_hz`. Keep the full list in `docs/value-names.md`, and let every driver use only names from that list.

Pitfall: off-by-one addresses. Many manuals number registers as 40001, 40002 and so on. Register 40001 usually means address 0 with function code 03. If every value looks shifted or nonsensical, check this first.

What you learn. Reading a technical manual precisely, and separating configuration from code.

Done when. The map for your meter covers every value the project needs, and each one has been checked against the meter's display.

4. Meter registry

The meter registry is one file that lists every installed meter and where it sits in the campus power tree; every other component reads its meter list from here.

Why we need it. Without one central list, meter details get copied into the collector, the dashboards and the reports, and the copies drift apart. The parent field also tells the system which meters should add up.

Fields per meter. ID, name, building, distribution board, driver, connection details, register map, parent meter, reading interval, active flag, device serial number.

Example (names and addresses invented):

```
meters:
  - id: gm-main-01
    name: Main incoming supply
    building: Main building
    board: Main low-voltage switchboard
    driver: modbus_tcp
    host: 10.0.5.21
    port: 502
    unit_id: 1
    register_map: maps/example_meter_x.yaml
    parent: null
    interval_s: 1
    active: true
```

```
    serial: SN-000123
  - id: gm-lab-02
    name: Lab wing feeder
    building: Lab wing
    driver: modbus_tcp
    host: 10.0.5.22
    port: 502
    unit_id: 1
    register_map: maps/example_meter_x.yaml
    parent: gm-main-01
    interval_s: 1
    active: true
    serial: SN-000456
```

With a third meter, `gm-lib-03`, also under the main meter, the tree is:

```
gm-main-01   Main incoming supply
├── gm-lab-02   Lab wing feeder
└── gm-lib-03   Library feeder
```

Design decision: the ID belongs to the measuring point, not the device. If the lab wing meter breaks and is replaced, the new device keeps `gm-lab-02`, so its history continues. The `serial` field records which hardware was installed when.

What you learn. Keeping a single source of truth, and modelling a physical installation as data.

Done when. All meters are listed, and the collector, health checker and dashboards take their meter lists only from this file.

5. Replay simulator

The replay simulator plays recorded meter data back as if it were coming live, so the team can build and test everything before getting network access to a real meter.

Why we need it. Campus network access can take weeks, and a real meter cannot produce faults on demand. The simulator removes both problems.

Two levels.

- **Level 1, MQTT replay:** reads the recorded file and publishes messages in the collector's format. Quick to build; tests everything from MQTT onwards.

- **Level 2, fake Modbus meter:** runs a small Modbus server that serves recorded values from its registers. The real collector and driver then read it, so the driver is tested too.

Speed factor. At 1× a week takes a week. At 60× a recorded week replays in about 2.8 hours, enough to test the weekly EN 50160 report in an afternoon.

Fault injection. Switches that deliberately break the data, so the health checker (component 9) can be tested:

```
replay.py --file data/samples/week_43.csv --speed 60 \
          --inject frozen --meter gm-lab-02 --start 10:00 --minutes 30
```

Other faults to inject: gaps, spikes (for example 9999 V), an energy counter jumping backwards, a clock 5 minutes off.

Example. A row from the recorded data (column names depend on your file):

```
timestamp,voltage_l1_v,current_l1_a,active_power_l1_kw
2026-10-12 08:00:00,231.4,40.1,8.82
```

becomes one MQTT message for `gm-main-01` with timestamp `2026-10-12T06:00:00Z`, because 08:00 Berlin summer time is 06:00 UTC.

What you learn. Test environments, reproducibility and time-zone handling.

Done when. A recorded week replays at 60× and ends up in the database with exactly the values of the source file.

6. Ingest service

The ingest service is the only component that writes measurements into the database: it listens on MQTT, checks every message, and stores the good ones in batches.

Why we need it. One gatekeeper keeps bad data out of the database and keeps the database independent of the meters.

MQTT topics. A clear topic structure makes subscribing simple:

```
gmcampus/<meter_id>/measurements    regular readings
gmcampus/<meter_id>/events          dips, swells, interruptions
gmcampus/<meter_id>/status          online, offline, errors
```

The ingest service subscribes to `gmcampus/+/measurements`. The `+` matches any single level, so new meters are picked up automatically.

Checks on every message.

- Valid JSON with `meter_id`, `ts` and `values`.
- The `meter_id` exists in the registry.
- The timestamp is not more than 1 minute in the future and not older than the collector's buffer limit.
- Every value is a number, and every name is in the agreed name list.

Messages that fail go to a reject log with the reason. They are never dropped silently.

Batch writes. Write every second or every 500 values, whichever comes first, instead of one database call per value.

Pitfall: duplicates. With MQTT QoS 1 ("at least once"), a message can arrive twice after a reconnect. Writes must be idempotent: the same meter and timestamp overwrite the existing row instead of adding a second one.

Example. A message from `gm-lab-99` arrives. That ID is not in the registry, so it is rejected and logged as "unknown meter". Either someone forgot to register a meter, or something is publishing that shouldn't be.

What you learn. Publish/subscribe messaging, input validation and idempotent writes.

Done when. Replaying the same hour twice leaves the row count unchanged, and malformed test messages appear in the reject log.

7. Data model

The data model decides how measurements are stored: which tables, which tags, how long raw data is kept, and which aggregates are computed.

Why we need it. A poor schema makes every later query slow or awkward. Raw data also grows fast: one meter with 50 values at one reading per second produces 4.3 million values a day, about 1.6 billion a year.

Choosing the database.

Criterion	InfluxDB	TimescaleDB
Query language	Its own languages, varying by version	Standard SQL

Criterion	InfluxDB	TimescaleDB
Joining with timetable, weather, registry	Harder	Easy, it is PostgreSQL
Retention and downsampling	Built in	Built in
Skill you take away	Time-series specific	SQL, used everywhere

The Development Setup Guide already runs PostgreSQL in Docker, and the team's architecture uses PostgreSQL with the TimescaleDB extension, so the right-hand column applies: standard SQL, plus time-series features such as automatic aggregates. Record this in `docs/decisions.md`.

Example schema (TimescaleDB, one row per meter and timestamp):

```
CREATE TABLE measurements (
    ts                TIMESTAMPTZ NOT NULL,
    meter_id          TEXT        NOT NULL,
    voltage_l1_v      DOUBLE PRECISION,
    current_l1_a      DOUBLE PRECISION,
    active_power_l1_kw DOUBLE PRECISION,
    thd_voltage_l1_pct DOUBLE PRECISION,
    -- one column per standard value name
    PRIMARY KEY (meter_id, ts)
);
SELECT create_hypertable('measurements', 'ts');
```

The primary key lets a repeated write overwrite the row (`INSERT ... ON CONFLICT ... DO UPDATE`) instead of doubling it. The alternative "narrow" layout, one row per value (`ts, meter_id, quantity, value`), needs no schema change for new value names but is larger and slower to query.

Aggregates.

Resolution	Used for	Kept
Raw (1 s)	Detail views, debugging	For example 90 days
1 min	Dashboards over days and weeks	Long term
10 min	EN 50160 evaluation	Long term

Resolution	Used for	Kept
15 min	Energy, peaks, forecasting	Long term

Store minimum, average and maximum for each interval, not just the average. An average hides exactly the spike you are looking for.

Pitfall: time zones. Store everything in UTC and convert to Berlin time only for display. On Sunday 25 October 2026 the clocks went back from 03:00 to 02:00, so that hour happened twice. With local timestamps its readings collide; in UTC they never do.

What you learn. Time-series data modelling, SQL, and storage trade-offs.

Done when. The schema is documented in `docs/`, and "average power per hour for one meter last week" returns in under a second.

8. Tests and deployment

Automated tests prove each part still works after every change, and one deployment file starts the whole system with a single command.

Why we need it. With three people changing code, something will break silently without tests. And a system that only one person can start does not survive the end of the project.

Unit tests check small pieces against known answers. Example with pytest, reusing the decoding example from component 2:

```
import pytest
from collector.decode import decode_float32

def test_float32_big_endian():
    assert decode_float32([0x4367, 0x3333], word_order="big") ==
pytest.approx(231.2, abs=1e-3)

def test_float32_swapped_words():
    assert decode_float32([0x3333, 0x4367], word_order="swapped") ==
pytest.approx(231.2, abs=1e-3)
```

Other good unit tests: each health-check rule on hand-made data, and the EN 50160 percentage on a small hand-calculated week.

Integration test. Start the broker, database and ingest service, replay one hour at high speed, then check that the expected number of rows arrived.

Deployment with Docker Compose. One file lists the services (broker, database, ingest, analytics, dashboard), and `docker compose up` starts them all. The collector runs separately on the edge device next to the meters.

Secrets. Passwords and keys go in a `.env` file listed in `.gitignore`. A `.env.example` with dummy values is committed instead.

Continuous integration. GitHub Actions runs all tests on every push and pull request; a failing test blocks the merge.

What you learn. Testing, containers and team workflows used in every software job.

Done when. A new person clones the repository, runs one command, and sees replayed data in a dashboard.

9. Data health checker

The health checker runs every 5 minutes (an APScheduler job) and gives each meter a green, yellow or red status with a plain reason, so nobody acts on a wrong number.

Why we need it. Every later component trusts the data. A frozen value or a reversed current transformer can make a report or an alert completely wrong without anyone noticing.

Rules with examples (thresholds are starting points, tuned later on real data):

Rule	Example	Status
Missing data	180 of 300 expected readings in 5 min (60%)	Yellow (50–90%); red below 50%
Frozen value	Voltage exactly 231.20 V for 10 minutes; real voltage always fluctuates	Red
Impossible value	Voltage 9999 V, frequency 0 Hz, power factor 1.3	Red
Counter backwards	Energy counter lower than 5 minutes ago	Red: meter reset or replaced

Rule	Example	Status
Wrong sign	Negative active power on a feeder with no generation behind it	Yellow: current transformer possibly mounted backwards
Clock drift	Meter time 4 minutes behind server time	Yellow

Balance check. Sub-meters should add up to their parent in the registry, minus a usual gap for cable losses and unmetered loads. Example: the main meter shows 120 kW; its three feeders show 52 + 31 + 25 = 108 kW, a gap of 12 kW (10%). If that gap is usually 8–10% and suddenly jumps to 35%, a sub-meter has probably dropped out. Compare the gap with its own history, not with zero.

Example output.

Meter	Status	Reason
gm-main-01	Green	All checks passed
gm-lab-02	Red	Voltage L1 frozen at 231.20 V since 10:00
gm-lib-03	Yellow	62% of expected readings in the last 5 min

The status is stored in the database, shown on the overview dashboard, and used by the EN 50160 report to leave out bad intervals.

What you learn. Data-quality engineering, often the most valuable part of a real monitoring system.

Done when. Every fault injected by the replay simulator is detected with the right reason, and clean replayed data stays green.

10. EN 50160 evaluator and weekly report

Every Monday this component checks the past week's voltage quality against EN 50160 limits and e-mails facility management a one-page pass/fail report.

Why we need it. Facility management needs a verdict, not charts. EN 50160 gives an accepted yardstick for what "good voltage" means.

Main limits for low-voltage supply (230 V nominal; check against the current edition through the university library):

Parameter	Limit	Share of time
Voltage magnitude, 10-min mean	230 V ± 10% (207–253 V)	95% of the week
Voltage magnitude, 10-min mean	+10% / −15% (195.5–253 V)	100% of the week
Frequency, 10-s mean	50 Hz ± 1% (49.5–50.5 Hz)	99.5% of the year
Frequency, 10-s mean	47–52 Hz	100% of the time
Long-term flicker Plt	≤ 1	95% of the week
Voltage unbalance, 10-min mean	≤ 2%	95% of the week
Total harmonic distortion THD (up to 40th)	≤ 8%	95% of the week
Individual harmonics, e.g. 3rd / 5th / 7th	≤ 5% / 6% / 5%	95% of the week

Worked example: the 95% rule. A week has 7 × 24 × 6 = 1,008 ten-minute values, so at most 50 may be outside the limit. Last week, voltage L1 was above 253 V in 12 intervals: 996 of 1,008 inside, which is 98.8%. Verdict: pass.

Three things to handle honestly.

- **Measuring point.** EN 50160 describes the supply at the connection point to the public grid. Inside the campus it serves as a reference benchmark, and the report should say so.
- **Measurement method.** The limits assume measurement per IEC 61000-4-30. A Class A meter gives comparable results; otherwise results are indicative. If the meter computes its own 10-min values, use those instead of averaging 1-s readings. Intervals containing a dip, swell or interruption are flagged and usually left out of the voltage statistics.
- **Completeness.** If less than, say, 90% of the week's data is valid (component 9), the verdict is "not enough data", not pass.

Example report.

Parameter	Limit	Result	Verdict
Voltage L1	207–253 V, 95%	98.8% inside	Pass

Parameter	Limit	Result	Verdict
THD voltage L2	⩽ 8%, 95%	100% inside	Pass
5th harmonic L3	⩽ 6%, 95%	99.6% inside	Pass
Flicker Plt L1	⩽ 1, 95%	91.7% inside	Fail
Data completeness	⩾ 90%	97.3%	OK

Plt is calculated every 2 hours, so a week has 84 values; 7 above 1 gives the 91.7% shown.

How it is built. Python computes the statistics, a template (for example Jinja2) fills an HTML or PDF page, and an APScheduler job sends it every Monday at 07:00 through the university mail server (ask IT for access).

What you learn. The standard itself, statistical evaluation and automated reporting.

Done when. For one replayed week, the report's percentage for at least one parameter matches a manual spreadsheet calculation exactly.

11. Dashboards a–d

Four dashboards answer four different questions, and each shows what its question needs and nothing more.

Dashboard	Question it answers	Main content
a. Overview	Is the campus normal right now?	Total power now, today's energy vs the same weekday last week, health status per meter, last week's EN 50160 verdict, open alerts
b. Meter detail	What is happening at this meter?	Meter dropdown from the registry; voltage, current and power per phase; harmonic spectrum as bars; flicker; event list
c. Compare meters	How do these buildings differ?	2–4 meters, one quantity, one time window, e.g. lab wing vs library on the same Tuesday
d. Compare time frames	How does now differ from then?	One meter, two periods overlaid, e.g. a lecture week vs the Christmas break

Design rules.

- Units on every axis and every number.
- The same colour for each phase everywhere (L1, L2, L3).
- Limit lines on charts, for example 207 V and 253 V on the voltage chart, so a breach is visible without reading values.
- Long time ranges read from aggregated tables. A month of 1-s data is 2.6 million points per value, far more than a screen can show.
- Times shown in Berlin time, stored in UTC.

Built with. Our own Django web app with Plotly charts, as in the team's architecture, instead of Grafana. Each dashboard is a Django page fed by our own views, so all four dashboards, the logins and the phone layout come from one codebase.

Example. A facility manager opens the overview on Monday morning. The lab wing shows an open alert: Sunday consumption was far above usual. One click opens the meter detail dashboard with the Sunday curve.

What you learn. Visualisation for a user who is not you.

Done when. Someone from facility management answers "is everything normal?" from the overview in under 10 seconds, without explanation.

12. Commissioning check and "How to add a meter" guide

Before a new meter's data is trusted, a script reads every value once and checks it for plausibility; a written guide lets people outside the team do this after the project ends.

Why we need it. Wrong register maps, swapped phases and reversed current transformers are common on day one. Catching them at commissioning costs minutes; finding them in a report months later costs trust.

Example run:

```
$ python commission.py --meter gm-lab-02

value                  reading     check
voltage_l1_v           231.2 V     OK (207-253 V)
current_l1_a            42.7 A     OK
power_factor_l1         0.95       OK
active_power_l1_kw      9.38 kW    OK (matches U x I x PF = 9.38 kW)
active_power_l2_kw     -8.71 kW    WARNING: negative on a consumer feeder
frequency_hz           49.99 Hz    OK

1 warning. Compare with the meter display before setting active: true.
```

The cross-check uses physics: per phase, active power ≈ voltage × current × power factor. Here 231.2 V × 42.7 A × 0.95 = 9.38 kW, so the L1 readings agree with each other.

The guide (`docs/how-to-add-a-meter.md`):

1. A qualified electrician (Elektrofachkraft) installs and wires the meter. Students do not work inside switchboards.
2. Get a fixed IP address from campus IT.
3. If the model is new, write its register map (component 3).
4. Add the meter to the registry with `active: false`.
5. Run the commissioning check, compare each value with the meter's display, and photograph the display.
6. Fix any warnings, then set `active: true`.
7. After 24 hours, confirm the health status is green.

What you learn. Commissioning practice from industrial automation, and writing for someone else.

Done when. A classmate outside the team adds a simulated meter using only the guide, without asking a question.

13. User management

Every person gets their own account with one of three roles, and access ends the moment the Admin deletes the account.

Why we need it. Shared logins make it impossible to know who changed what, and people who leave keep access. Roles also stop a viewer from changing settings by accident.

Permission matrix (a proposal for the team to adjust):

Action	Admin	Operator	Viewer
View dashboards and reports	Yes	Yes	Yes
Download data and reports	Yes	Yes	No
Acknowledge alerts, add notes	Yes	Yes	No
Edit dashboards and alert thresholds	Yes	Yes	No
Edit the meter registry	Yes	No	No
Create and delete users	Yes	No	No

Principles.

- One personal account per person; no shared "facility" login.
- Least privilege: give the lowest role that does the job.
- Passwords are stored only as hashes, never as plain text; Django does this by default. Whether university accounts (single sign-on) replace our own is an open question for campus IT.
- An audit log records who did what and when.

Example. A student assistant joins facility management for one semester and gets a Viewer account. When the contract ends, the Admin deletes it; the next page load is refused and open sessions end.

Built with. Django's built-in users and groups: one group per role (Admin, Operator, Viewer), each with its permissions. Lockout after repeated failed logins comes from django-axes, already in the package list. The extra protections for a login reachable from

the internet (HTTPS, two-factor login, session timeout, admin pages only inside the campus network) are in the Phone Access and Login doc.

What you learn. Authentication, authorisation and basic security practice.

Done when. A test per role proves what it can and cannot do; for example, a Viewer's attempt to change a threshold is refused.

14. Waste finder

The waste finder compares the energy used when buildings are empty (nights, weekends, holidays) with the energy used when they are busy, and turns the difference into a ranked list of savings to investigate.

Why we need it. Energy used in an empty building is the easiest energy to save. Without context data, nobody can tell whether 40 kW at 03:00 is normal.

Context data to add.

- Lecture timetable: which buildings have lectures in which hours.
- Public holidays in NRW and semester breaks.
- Outdoor temperature, for example from the open data of the German weather service (DWD).

Method.

1. Label every hour as occupied or unoccupied.
2. Compute each building's base load, for example the median power between 01:00 and 04:00.
3. Compare base load with average power in busy hours.
4. Estimate avoidable energy: unoccupied hours × the part of the base load that could be switched off.

Example (invented numbers). The lab wing draws 40 kW at 03:00 on a Sunday and 95 kW on a Tuesday at 11:00, so its night load is 42% of its busy load. If facility management confirms that 15 kW of it could be switched off for 100 unoccupied hours a week, that is 1,500 kWh a week. At an assumed 25 ct/kWh, that is about €375 a week.

Clustering. Treat each day as a shape of 96 quarter-hour values, scaled so only the shape counts. k-means then groups the days; you would expect groups such as lecture days, weekends and holidays. A day far from its group is odd: a Saturday shaped like a lecture day suggests a ventilation schedule left on.

What you learn. Joining data sources, feature engineering, unsupervised learning, and turning analysis into money.

Done when. The team shows facility management a top-5 list of waste periods with kWh estimates, and they confirm or explain each one.

15. Alert engine

The alert engine warns when a meter behaves unlike its own past, and every alert says what is unusual and by how much.

Why we need it. Nobody watches dashboards all weekend. Ventilation running all Sunday should be noticed on Sunday, not at the end of the month.

Stage 1: hour-of-week baseline. A week has 168 hours. For each meter and each of these 168 slots, the baseline is the median of the same slot over the last 6 weeks, with a spread such as the interquartile range. An alert fires when a value stays above baseline + 3 × spread for 2 hours in a row. Holidays get their own baseline or are left out, otherwise Christmas looks like a fault.

Example: on Sunday at 14:00 the lab wing's baseline is 22 kW with a spread of 3 kW, so the threshold is 31 kW. It measures 41 kW for 3 hours, so an alert fires:

```
[Warning] Lab wing (gm-lab-02)
Sunday 14:00-17:00: 41 kW, usually 22 kW (±3 kW) at this hour.
Above normal for 3 hours. Possible cause: ventilation or heating schedule.
```

Avoiding alert fatigue. Group repeated alerts into one, add a cooldown (no repeat within 6 hours), use severity levels, and let Operators acknowledge alerts. Too many alerts are as useless as none.

Stage 2: Isolation Forest. A machine-learning model trained on hourly features: mean and maximum power, power factor, THD, hour, weekday, occupancy flag and outdoor temperature. It gives each hour an anomaly score. To keep the "reason" requirement, the alert names the feature that differs most from normal. The interesting experiment: does the model find anything stage 1 misses, and at what cost in false alerts?

Delivery. E-mail first; Microsoft Teams if university IT allows the connection.

What you learn. Baselines before machine learning, anomaly detection, and designing alerts people actually read.

Done when. On replayed history with injected anomalies, every anomaly triggers an alert and clean weeks produce at most one or two false alerts.

16. Event detective

The event detective lines up power-quality events from several meters by time and uses the meter tree to tell whether a disturbance came from the public grid or from inside the campus, and from which branch.

Why we need it. After a dip trips a machine, the first question is whose problem it is. If it came from the grid, it is a matter for the network operator; if it came from inside, the campus can fix it.

Event types (EN 50160 terms):

- **Dip:** voltage falls below 90% of the reference voltage but stays above 5%, for 10 ms up to 1 minute.
- **Swell:** voltage rises above 110%.
- **Interruption:** voltage falls below 5%.

Meters with power-quality functions usually log each event with start time, duration, depth and phase.

Logic.

1. Collect events from all meters.
2. Group events whose start times lie within a small window, for example 100 ms; the window depends on how closely the clocks agree.
3. Check which meters in the registry tree saw the group. If the main meter and all feeders saw it, the cause is upstream, in the public grid. If only one branch saw it, the cause is inside that branch.

Example 1, grid event. At 10:42:13.250 the main meter records a dip to 78% for 80 ms, and the lab wing and library meters record the same dip within 20 ms. Verdict: from the public grid.

Example 2, campus event. At 14:05:02.100 only the lab wing records a dip to 85% for 120 ms; the main meter records nothing. Verdict: inside the lab wing, for example a large motor or compressor starting.

Requirements. At least two meters in a parent–child relationship, clocks synchronised by NTP (or PTP) to within a few milliseconds, and meters that log events with millisecond timestamps. With only one meter, this component cannot be built.

What you learn. Event correlation, time synchronisation, and fault analysis in power distribution.

Done when (stretch). Simulated events placed on two meters are correctly classified as grid or campus events.

17. Peak forecast

The peak forecast predicts tomorrow's load in 15-minute steps and warns a day ahead if a new yearly peak looks likely, so loads can be shifted in time.

Why we need it. For large consumers in Germany, part of the grid fee depends on the highest 15-minute average power of the year. One bad quarter-hour can set that cost for the whole year. Ask facility management how the campus tariff is structured.

Features. Load at the same time yesterday and last week, hour, weekday, holiday and semester flags, timetable occupancy, and the temperature forecast.

Method.

1. Start with a naive baseline: tomorrow equals the same weekday last week.
2. Train LightGBM on the features above.
3. Keep LightGBM only if it beats the baseline on weeks it has never seen, for example training on two months and testing on the next one.
4. Measure the error in kW overall, and separately on each day's maximum, because the maximum is what matters here.

Example (invented numbers). The forecast for tomorrow peaks at 410 kW at 11:15, and this year's highest value so far is 420 kW. The forecast is less than 3% below it, so the system warns: "Possible new yearly peak tomorrow around 11:15." Facility management could then delay EV charging or move a large test run.

Data needed. Months of history, ideally a full year to cover the seasons, which is why this is a stretch goal. Ask facility management whether the utility's 15-minute load data for the main connection already exists; that could unlock this component early.

What you learn. Time-series forecasting, gradient boosting, and honest evaluation against a baseline.

Done when (stretch). The model beats the naive baseline on held-out weeks, both overall and on daily maxima.

18. Mobile access

The whole system is built as a responsive web app: every screen facility management uses opens in a phone's browser, so no separate mobile app is needed.

Decision (28 Oct 2026): the phone app is a PWA. No native app and no app-store listing; the same Django code serves PC and phone.

Why we need it. Facility staff are rarely at a desk; they work in plant rooms and at switchboards. A Sunday alert is read on a phone, and the commissioning check is done standing at the meter.

Why not a native app. Separate Android and iOS apps mean a second codebase, app-store accounts and reviews, and double the testing, which is too much for this project. A responsive web app reaches every phone from the same code. Installed to the home screen as a Progressive Web App (PWA), it opens like an app.

What works on the phone.

Part	On the phone
11a Overview	One column: campus power, health status per meter, open alerts
11b Meter detail	One chart per screen, scroll between them
11c, 11d Comparisons	Viewable; easier on a laptop
15 Alerts	Arrive on the phone; acknowledge with one tap
10 Weekly report	E-mail readable on the phone, with the PDF attached
12 Commissioning check	Run at the switchboard: each reading with a "matches display?" tick
13 Login	Same personal accounts and roles as on the laptop
Registry editing, user admin	Laptop only, on purpose, to avoid mistakes on a small screen

Design rules for small screens.

- Design the phone layout first, then widen it for the laptop.
- One column, large tap targets (about 44 px), nothing that only shows on mouse hover.
- Verdict first (green, yellow, red; pass or fail), details one tap away.
- Send aggregated data to charts, not raw 1-s values, so pages load fast on mobile data.

Getting alerts to the phone. Start with e-mail, which every phone already has. Next steps: Microsoft Teams messages if university IT allows them, or web push notifications from the PWA (on iPhones these work once the app is added to the home screen).

Access from outside the campus. Still open; it is on the agenda for the next team meeting in the Phone Access and Login doc. Option A: only on the campus network or through the university VPN. Option B: the web app is reachable from the internet through the university firewall, on port 443 only. It then needs HTTPS, account lockout, strong passwords, two-factor login and a session timeout. Meters, database and MQTT broker stay internal either way, and because the app is read-only and cannot switch any equipment, option B is easier to argue for with campus IT.

How it fits the chosen tools. The phone or PC only shows the data; all the work happens on the server. The Django app sends pages that adapt to the screen width, and a web-app manifest plus a small service worker make it installable as a Progressive Web App, with its own icon, full-screen view and push alerts. Nothing runs on the phone itself: no Python, no Docker, no setup.

What turns the Django app into a PWA.

1. A manifest file (`manifest.json`) with the app's name, icons (192 and 512 px), start page, theme colour and `"display": "standalone"`, so it opens full-screen like an app.
2. A service worker (`sw.js`), a small script the phone keeps in the background. It caches the app's frame for a fast start, shows an offline page when there is no signal, and receives push alerts.
3. Two additions to Django's base template: a link to the manifest, and a short script that registers the service worker.
4. HTTPS. Phones only install a PWA from a secure address. `localhost` counts as secure, so development on a PC works without a certificate.

Example manifest (name, colour and paths are suggestions):

```
{
  "name": "GM Campus Power",
  "short_name": "GM Power",
  "start_url": "/",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#0b5cab",
  "icons": [
    {"src": "/static/icons/icon-192.png", "sizes": "192x192", "type":
"image/png"},
    {"src": "/static/icons/icon-512.png", "sizes": "512x512", "type":
"image/png"}
  ]
}
```

Checking it works. In Chrome on the PC, Developer Tools → Application → Manifest shows whether the app is installable and what is missing. On a real phone it can be installed once the campus server runs with its HTTPS certificate.

Push alerts. Web Push needs a key pair (VAPID keys) on the server and a library such as pywebpush, which is not yet in `requirements.txt`; add it when the alert engine (component 15) is built. On iPhones, push works only after the app has been added to the home screen.

What you learn. Responsive web design, PWAs, notifications and secure remote access.

Done when. On a phone over mobile data (through the access route IT approves), a facility manager sees the overview, receives an alert and acknowledges it in under 30 seconds, and the commissioning check can be completed standing at a meter.

19. Data export and API

People with the right role can download chosen measurements as a CSV file, or fetch them from a small read-only web address (an API), without direct access to the database.

Why we need it. "Export data for research" is a Should requirement in the team's Vision doc. Campus data is useful for theses and other courses, and a controlled export keeps the database safe from ad-hoc queries.

How it works.

1. A form in the web app: pick meters, values, time range and resolution (1, 10 or 15 min).

2. The app builds the CSV from the aggregated tables, so even a year of data stays a manageable file.
3. The same query is offered as a read-only API address for scripts. A plain Django view returning JSON is enough; no extra package is needed:

```
GET /api/v1/measurements?meter=gm-lab-02&value=active_power_l1_kw
    &from=2026-11-01&to=2026-11-30&resolution=15min
```

4. Every export is logged: who downloaded what, and when.

Rules. Only Operators and Admins may export, matching the permission matrix in component 13. Timestamps are in UTC with the time zone stated in the file header. Very large raw exports are refused or split.

Example. A student writing a thesis on campus load profiles downloads November's 15-minute power for three buildings: 3 meters × 30 days × 96 values = 8,640 rows.

What you learn. API design, query limits and access control.

Done when. A teammate downloads one month for one meter as CSV, and its total energy matches what the dashboard shows.